FOR BEGINNERS

Web Workers

The browser trick that makes your app feel super fast

🍳 *Think of it like hiring a helper in the kitchen so you can keep cooking without stopping.*

97%	2010	0	Free!
Works in all browsers	Available since	Extra installs needed	Built into the browser

01 — The Problem, Simply Explained

Why does my app freeze? 🤔

🍳 Imagine you're cooking and someone asks you to count all the rice grains in a bag.

You stop cooking. You count. The food burns. Then you answer.

❌ Your browser today

It has ONE worker (the main thread) doing EVERYTHING:
→ Show the page
→ Listen to your clicks
→ Run heavy calculations

If one task takes too long — everything else WAITS.

✅ With Web Workers

You hire a helper (Web Worker) for the heavy task.

The main thread keeps the UI smooth.
The worker crunches data in the background.

Both work at the same time!

The main thread = your browser's only worker 👷

MAIN THREAD

- 🖱️ Handle your clicks
- 📜 Update the DOM
- 🎨 Paint the screen
- 🎬 Play Animations
- ⚙️ Run your JavaScript
- ⌨️ Read keyboard

😫 If ONE of these tasks takes too long, ALL the others are stuck waiting. That's the freeze!

A background helper that runs separately 🤷

A Web Worker is a separate JavaScript file that runs in its own thread — completely independent from your main app.

Its own thread

Runs completely parallel — not in competition with your UI

Can't touch the DOM

It can't update HTML — that stays in the main thread's job

Talks via messages

You send it data, it sends results back — like texting a helper

Nothing to install

Built into every modern browser since 2010. Zero setup needed.

3 simple steps — send, process, receive 📨

1

📤 Main thread sends heavy data to the Worker

```
worker.postMessage(bigArray);   ← You literally just do this one line
```

2

⚙️ Worker does the heavy job in the background

```
self.onmessage = ({data}) => { self.postMessage(data.sort(fn)); }
```

> 👂 *While the Worker is busy — your UI is still 100% responsive. Scroll, click, animate. No freeze!*

3

📥 Worker sends result back. Main thread shows it.

```
worker.onmessage = (e) => renderTable(e.data);   ← Done!
```

That's the whole pattern. 3 lines across 2 files. No library needed.

Real situations every developer faces 💻

Big data tables

Your dashboard loads 10,000 rows and the page hangs for 2 seconds? Worker fixes this.

File uploads

User uploads a 20MB CSV? Parsing it on the main thread freezes the upload progress bar.

Encrypting data

Securing passwords or sensitive data before sending to your API? That math is expensive — use a Worker.

Image editing

Adding filters, resizing, or compressing images in-browser? Workers handle this silently.

Search & filter

Searching through thousands of products in real-time? Move that logic to a Worker.

AI in browser

Running ML models like face detection or text classification directly in the browser.

Pick the right tool for the job 🛠️

START HERE

Dedicated Worker

Works for ONE script only. This is the most common type and what you'll use 99% of the time as a beginner.

- → Sort a big array
- → Parse uploaded file
- → Heavy number crunching

ADVANCED

Shared Worker

Shared across multiple browser tabs or scripts. Useful when you need the same worker to serve multiple pages.

- → Sync data across tabs
- → Shared cache layer
- → Multi-tab coordination

NETWORK LAYER

Service Worker

Intercepts network requests. Powers offline apps and smart caching. This is what makes Progressive Web Apps work.

- → Offline mode
- → Cache assets
- → Push notifications

Web Workers are great — but not perfect ⚠️

Can't touch the page (DOM)

Analogy: It's like a chef in a separate kitchen — they cook, but can't serve the food directly. You (main thread) do the serving.

✅ Fix: The Worker sends the result back, then YOU update the page.

Data is copied, not shared

Analogy: Imagine photocopying a 500-page document to send to your helper. It's safe but takes time if the document is huge.

✅ Fix: Use Transferable Objects for large data — they move instead of copy (advanced tip).

Harder to debug

Analogy: Like troubleshooting two phones talking to each other — you need to check both sides.

✅ Fix: Chrome DevTools has a dedicated Sources tab section for Workers. Start there.

Don't use for tiny tasks

Analogy: Hiring a full-time assistant to shred one piece of paper is overkill.

✅ Fix: Use Workers only when the computation takes > ~16ms and causes visible UI lag.

Should I use a Web Worker? Here's your cheat sheet ✅

✅ YES — Use a Worker

- ⏱️ Task takes more than ~16ms to run
- 📊 Sorting or filtering thousands of rows
- 📂 Parsing a large uploaded file
- 🔒 Running encryption/hashing algorithms
- 🤖 Running an AI/ML model in the browser
- 🔍 Real-time search through large local data

❌ NO — Keep it on main thread

- 🔘 Adding one item to a short list
- 🖱️ Simple button click handlers
- 🎨 Updating CSS or colors
- 📝 Form validation on a few fields
- 🔔 Making a regular API call (use async/await)
- ⚡ Any task that finishes instantly

Once you're comfortable — explore these 🚀

01 Transferable Objects

Instead of copying big files, you can TRANSFER them (like handing over a USB drive instead of duplicating it). Way faster.

Medium

02 OffscreenCanvas

Draw graphics and animations entirely inside a Worker. Take rendering off the main thread completely.

Medium

03 Worker Pools

Create a team of 4–8 workers and give them tasks in parallel — like multiple chefs in the kitchen.

Advanced

04 SharedArrayBuffer

True shared memory between threads — multiple workers reading/writing the same data at once.

Advanced

05 Comlink (Google)

A tiny library that hides all the postMessage boilerplate. Workers feel like regular async functions.

Easy add-on

What You've Learned Today

- The browser has ONE main thread — it does everything
- Heavy tasks block that thread and freeze your UI
- Web Workers run heavy code in a SEPARATE background thread
- They communicate via postMessage — simple send & receive
- 3 types: Dedicated (start here), Shared, and Service Workers
- Use them when a task takes >16ms and causes visible lag

"Framework knowledge helps you build apps faster. Browser fundamentals help you build better apps."